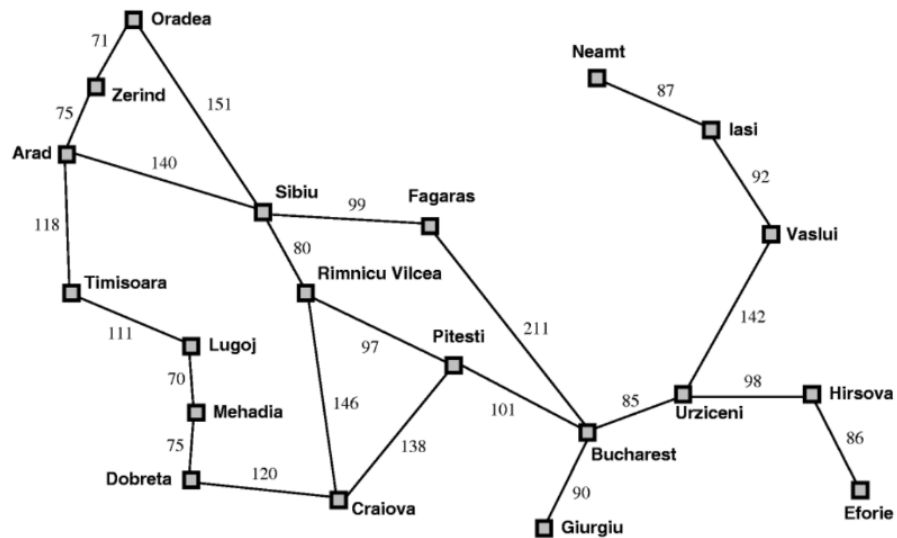


Nafil Fadillah
20123958
3KB02

Pertemuan III

Studi Kasus

Dalam studi kasus kali ini, akan melakukan pencarian jarak terdekat dari kota Arad ke Bucharest.



```
In [1]: # Define graph with tipe data dictionary
```

```
graph = {
    'A': ['Z', 'S', 'T'],
    'Z': ['O'],
    'O': ['S1'],
    'S1': ['F', 'R'],
    'F': ['B'],
    'B': [],
    'R': [],
    'S': ['O1', 'F1', 'R1'],
    'O1': [],
    'F1': ['B1'],
    'B1': [],
    'R1': ['C', 'P'],
    'C': [],
    'P': [],
    'T': ['L'],
    'L': ['M'],
    'M': ['D'],
    'D': []
}
```

```
In [2]: visited = set() # Set to keep track of visited nodes of graph.
```

```
In [3]: def dfs(visited, graph, node): # Function for dfs
```

```
    if node not in visited:
        print (node)
        visited.add(node)
        for neighbour in graph[node]:
            dfs(visited, graph, neighbour)
```

```
In [4]: print("Following is the Depth-First Search")
```

```
Following is the Depth-First Search
```

```
In [5]: dfs(visited, graph, 'A')
```

A
Z
O
S1
F
B
R
S
O1
F1
B1
R1
C
P
T
L
M
D

Laporan Analisis Implementasi Algoritma Depth-First Search (DFS)

1. Deskripsi Studi Kasus

Studi kasus ini bertujuan untuk menemukan rute perjalanan dari kota Arad menuju kota Bucharest menggunakan peta keterhubungan kota di Rumania. Setiap kota direpresentasikan sebagai sebuah *node* (titik), dan jalan antar kota direpresentasikan sebagai *edge* (garis) yang memiliki bobot jarak tertentu.

2. Metodologi: Depth-First Search (DFS)

Algoritma yang digunakan dalam implementasi kode Python tersebut adalah Depth-First Search (DFS). DFS adalah algoritma penelusuran struktur data graf yang bekerja dengan prinsip prioritas kedalaman.

Karakteristik Utama DFS:

- Penelusuran Mendalam:** Algoritma akan menelusuri satu cabang hingga mencapai titik terdalam (atau tujuan) sebelum melakukan *backtracking* (kembali ke persimpangan sebelumnya) untuk mencoba cabang lain.
- Struktur Data:** Secara implisit menggunakan struktur *Stack* (LIFO - Last In First Out) dalam proses rekursinya.
- Eksplorasi Tetangga:** Urutan kunjungan sangat bergantung pada urutan pendefinisian tetangga dalam struktur data *dictionary*.

3. Penjelasan Implementasi Kode

Berdasarkan kode program yang dilampirkan, berikut adalah fungsi dari masing-masing bagian:

- graph (Dictionary):** Berfungsi untuk merepresentasikan peta (Adjacency List). Contohnya, 'A': ['Z', 'S', 'T'] menunjukkan bahwa dari kota A (Arad), sistem dapat bergerak ke kota Z, S, atau T.
- visited = set():** Variabel ini berfungsi untuk menyimpan daftar kota yang telah dikunjungi. Hal ini krusial untuk mencegah terjadinya *infinite loop* (perulangan tak terbatas) jika terdapat jalur yang berputar (cycle).

- c. **Fungsi dfs(visited, graph, node):** 1. Melakukan pengecekan apakah kota saat ini sudah dikunjungi. 2. Jika belum, kota dicetak dan dimasukkan ke dalam daftar visited. 3. Secara rekursif memanggil fungsi yang sama untuk setiap tetangga dari kota tersebut sesuai urutan di dalam *dictionary*.

4. Analisis Hasil Eksekusi

Berdasarkan hasil *output* program: **A -> Z -> O -> S1 -> F -> B1...**

Dapat disimpulkan bahwa sistem bergerak dari Arad (A), memilih jalur pertama yaitu Zerind (Z), lalu ke Oradea (O), hingga mencapai Sibiu (S1) dan seterusnya. Jalur ini diambil karena algoritma DFS selalu memprioritaskan penyelesaian satu jalur hingga tuntas sebelum mempertimbangkan jalur alternatif (seperti jalur langsung Arad ke Sibiu atau Arad ke Timisoara).

5. Kesimpulan

Implementasi DFS dalam studi kasus ini berhasil menemukan jalur menuju target (Bucharest). Namun, perlu dicatat bahwa DFS tidak menjamin penemuan jarak terdekat (rute terpendek), melainkan hanya menjamin penemuan jalur berdasarkan urutan eksplorasi mendalam. Untuk mendapatkan rute dengan total jarak minimal, diperlukan algoritma optimasi lain seperti *Breadth-First Search (BFS)* atau *A-Star (A*)*.